Assignment-Regression Algorithm

Problem Statement - **TO PREDICT THE INSURANCE CHARGES**.

Dataset Info - **1338** rows × **6** columns

Data Pre-processing - By using the pandas predefined function pandas.get_dummies(), we can convert a categorical column in the dataset to integer type.

Multiple Linear regression algorithm - R2 Value is **0.7862**

Support Vector Machine

#	Hyper Param C	Linear	Poly	RBF	Sigmoid	precomputed
		R2 Value				
1	0.1	0.1256	0.0070	0.3853	0.1213	Can do only for square matrix
2	0.5	0.7698	0.7208	0.3853	0.1213	
3	10	0.6938	0.7565	0.7315	**0.7994**	
4	50	0.6773	0.6919	0.6800	0.7555	
5	100	0.6789	0.6789	0.6836	0.7395	

Decision Tree

#	Criterion	Splitter	Max Value	R2 Value
1	squared_error	Best	None	**0.6981**
2	squared_error	Best	sqrt	**0.7434**
3	squared_error	Best	log2	**0.7268**
4	squared_error	Random	None	**0.6752**
5	squared_error	Random	sqrt	**0.6077**
6	squared_error	Random	log2	**0.4568**
7	friedman_mse	Best	None	**0.6880**
8	friedman_mse	Best	sqrt	0.7976
9	friedman_mse	Best	log2	**0.7452**
10	friedman_mse	Random	None	**0.6905**
11	friedman_mse	Random	sqrt	**0.6720**
12	friedman_mse	Random	log2	**0.6862**
13	absolute_error	Best	None	**0.6897**
14	absolute_error	Best	sqrt	**0.7249**
15	absolute_error	Best	log2	**0.7600**
16	absolute_error	Random	None	**0.7465**
17	absolute_error	Random	sqrt	**0.6367**
18	absolute_error	Random	log2	**0.7085**
19	poisson	Best	None	**0.6795**
20	poisson	Best	sqrt	**0.6864**
21	poisson	Best	log2	**0.7402**

22	poisson	Random	None	**0.7294**
23	poisson	Random	sqrt	**0.6838**
24	poisson	Random	log2	**0.7150**

Random Forest

#	*n_estimators*	Criterion	Max Value	R2 Value
1	50	squared_error	None	**0.8251**
2	50	squared_error	sqrt	**0.8210**
3	50	squared_error	log2	**0.8210**
4	100	squared_error	None	**0.8226**
5	100	squared_error	sqrt	**0.8212**
6	100	squared_error	log2	**0.8212**
7	50	friedman_mse	None	**0.8251**
8	50	friedman_mse	sqrt	0.8210
9	50	friedman_mse	log2	**0.8210**
10	100	friedman_mse	None	**0.8226**
11	100	friedman_mse	sqrt	**0.8212**
12	100	friedman_mse	log2	**0.8212**
13	50	absolute_error	None	**0.8409**
14	50	absolute_error	sqrt	**0.8301**
15	50	absolute_error	log2	**0.8348**
16	100	absolute_error	None	**0.8406**
17	100	absolute_error	sqrt	**0.8348**
18	100	absolute_error	log2	**0.8348**

19	50	poisson	None	**0.8261**
20	50	poisson	sqrt	**0.8293**
21	50	poisson	log2	**0.8293**
22	100	poisson	None	**0.8236**
23	100	poisson	sqrt	**0.8263**
24	100	poisson	log2	**0.8263**

Result

#	Algorithm	Best result params	R2 Value
1	Multiple Linear Regression	-	0.7862
2	SVM	c=10, kernal=Sigmoid	0.7994
3	Decision Tree	criterion='friedman_mse',splitter='best',max_features='sqrt'	0.7976
4	Random Forest	n_estimators=50,random_state=0,criterion='absolute_error',max_features=None	0.8409

For this problem **Random Forest Regressor** gives more accuracy, so we can use this algorithm for better output R2 value is **0.8409**